UNIVERSIDAD AUTÓNOMA DE AGUASCALIENTES

CENTRO DE CIENCIAS BÁSICAS

“REPORTE DE DIAGNÓSTICO Y DESPLIEGUE DE PROTOTIPO. EVALUACIÓN PARA SEGUNDO PARCIAL”

Proyecto: “Classificatory de Torneos – Sistema ArenaUAA”

Integrantes:

Gildardo Alejandro Avendaño Nuñez

José Emilio Durán Ramos

Heidi Romina López Estrada

Johanna Karol Reyes Macías

PROYECTOS TECNOLÓGICOS

Lic. en Informática y Tecnologías Computacionales

5°A

Profesor: CARLOS ARGELIO AREVALO MERCADO

Fecha: 03/11/2026

1. Datos Generales

Nombre del equipo: HUNTRIX

Tema/Nombre del proyecto: "Classificatory de Torneos – Sistema ArenaUAA"

Liga al repositorio de GitHub: https://github.com/Gildardo00/despliegue-proyecto.git

2. Estado actual del prototipo

El diagnóstico del prototipo ArenaUAA se realizó mediante la revisión del código fuente y el seguimiento de su proceso de despliegue. Actualmente, la aplicación se encuentra desplegada en Microsoft Azure for Students, utilizando Apache Tomcat 10 y MySQL 8. Aunque se ha completado su publicación en el servidor, el prototipo todavía presenta funcionalidades parciales y aspectos pendientes de mejora, los cuales se describen a continuación.

- **Funciona:** inscripción pública con validación de campos obligatorios (RF-01), filtro de lenguaje ofensivo en usuarios, equipos y generador (RF-03), emparejamiento por siembra #1 vs último (RF-05a), Round Robin aleatorio sin repetir pares (RF-05b), equipos aleatorios balanceados con diferencia máxima de 1 (RF-06), soporte multidisciplina (RF-08), login con sesión, CRUD de disciplinas, equipos, jugadores, rondas, jornadas, partidos y participaciones, dashboard y gráfica.
- **Parcial:** clasificación (RF-02) y brackets (RF-04) no exigen que las inscripciones estén cerradas, y solo se genera la Ronda 1; el cierre por fecha (RF-07) se guarda, pero nada lo evalúa; la integridad (RNF-06) no evita jugadores duplicados; el acceso (RNF-03) usa sesión, sin VPN, restricción de red ni roles.
- **Pendiente:** rondas siguientes del bracket, cifrar contraseñas (hoy en texto plano), disciplinas individuales como judo o ajedrez, agrupar el Round Robin en jornadas reales, respaldo automático (RNF-05) y pruebas de carga y disponibilidad (RNF-01, 02, 04).

3. Stack Tecnológico

El prototipo ArenaUAA está desarrollado en Java, con compatibilidad de compilación para Java 11 y ejecución en Java 17 en el servidor. Utiliza Jakarta Servlets 6.0 para gestionar las peticiones HTTP y JavaServer Pages (JSP), HTML y CSS para la interfaz de usuario.

Para la persistencia de datos se utiliza MySQL 8, mediante conexiones JDBC con MySQL Connector/J 8.4.0. El proyecto emplea Apache Maven para la gestión de dependencias y compilación, Apache Tomcat 10 como servidor de aplicaciones y NetBeans como entorno de desarrollo.

La infraestructura de despliegue se encuentra alojada en Microsoft Azure for Students, mediante una máquina virtual con Ubuntu Server 24.04 LTS.

4. Arquitectura elegida y Justificación

Para el desarrollo del prototipo ArenaUAA se eligió la arquitectura por capas, basada en el patrón Modelo-Vista-Controlador (MVC), con el propósito de organizar el sistema y separar las responsabilidades de sus componentes. Esta arquitectura se aplica de la siguiente manera:

- Modelo: está compuesto por las clases de entidades y DAO, encargadas de representar la información y gestionar el acceso a la base de datos MySQL.
- Vista: utiliza JSP, HTML y CSS para presentar la información del sistema y permitir la interacción con los usuarios.
- Controlador: está implementado mediante Servlets, que reciben las solicitudes, gestionan las operaciones y coordinan la comunicación entre las vistas y el modelo.

Justificación de la elección: Se eligió MVC porque ArenaUAA es un prototipo universitario que necesita gestionar inscripciones, equipos, disciplinas, participantes y torneos de forma organizada. La separación de responsabilidades permite desarrollar, identificar errores y modificar cada componente con mayor facilidad, sin tener que modificar directamente las demás partes del sistema. Asimismo, favorece el mantenimiento y la incorporación de nuevas funcionalidades conforme avance el proyecto.
Se consideró más adecuada que otras arquitecturas, como microservicios o arquitectura orientada a eventos, porque el prototipo tiene un alcance limitado, contempla aproximadamente 50 usuarios simultáneos y dispone de recursos económicos y técnicos reducidos. Estas alternativas introducirían una mayor complejidad de desarrollo, comunicación y despliegue que actualmente no se justifica. Por ello, MVC ofrece una solución más sencilla,

organizada y acorde con las necesidades reales del proyecto, manteniendo una estructura que facilita su crecimiento futuro.

5. Proveedor de despliegue

Criterios de decisión: costo cero o muy bajo (restricción de presupuesto del cliente), compatibilidad con Servlets/JSP + Tomcat + MySQL sin reescribir código, y que la cuenta pudiera activarse con el correo institucional.

Opción	Criterio / resultado	Decisión
Oracle Cloud Free Tier	VM ARM «Always Free»; cumple costo cero y corre Tomcat + MySQL.	Primera opción considerada; el equipo buscó otras alternativas debido a que hubo problemas con la cuento asignada al proveedor y se decidió buscar otra alternativa más eficaz y optima para el desarrollo de despliegue.
Google Cloud	Exigió un prepago real de MXN 600.	Descartado (rompe la restricción de presupuesto)
Railway, Render, Koyeb, Alwaysdata, Fly.io	Sin plan gratuito viable para Tomcat + MySQL.	Descartados
Firebase	No ejecuta Servlets/JSP y es NoSQL; obligaría a reescribir casi todo.	Descartado (incompatible)
Azure for Students	Aceptó el correo UAA; USD 100 de crédito por 365 días; VM Ubuntu con Tomcat 10 y MySQL 8.	**ELEGIDO**

Paso del proceso: Se realizó el despliegue del prototipo ArenaUAA utilizando Microsoft Azure for Students. Para ello, se configuró una máquina virtual denominada «Organizador-Torneos», ubicada en Canadá Central, con Ubuntu Server 24.04 LTS ARM64, 2 vCPU y 1 GiB de RAM. Se instalaron OpenJDK 17, Apache Tomcat 10 y MySQL 8, además de configurar 2 GB de memoria swap para optimizar el funcionamiento del servidor. Posteriormente, se integró la base de datos y se publicó la aplicación en Apache Tomcat

Liga a Aplicación Desplegada: http://20.151.215.246:8080

Usuario: admin - Contraseña: admin123

6. Problemas técnicos encontrados

- **Cuotas y políticas del proveedor:** Google Cloud pidió prepago; en Azure los tamaños B1s, B1ms y B2s dieron NotAvailableForSubscription, y una política de la suscripción limitó las regiones (North Central US falló con RequestDisallowedByAzure).
- **Tamaño de VM:** el tamaño por defecto (≈ USD 70/mes) no estaba disponible; el B2pts_v2 exigió cambiar la seguridad de «inicio seguro» a Estándar.
- **Poca memoria (1 GiB):** se agregó swap de 2 GB y se limitó Tomcat (-Xmx384m) y MySQL (buffer pool de 128 MB); la primera visita a cada JSP puede tardar.
- **Credenciales fijas en el código:** Conexion.java trae root/admin y no lee variables de entorno, así que usar un usuario seguro obliga a editar y recompilar el .war.
- **Errores operativos:** se trabajó primero sobre el proyecto terminado y no sobre el prototipo (ya corregido), Maven no estaba en el PATH y un apt install falló por pegar varios comandos juntos.

7. Evidencia de contribución

Liga a Contributors: https://github.com/Gildardo00/despliegue-proyecto/graphs/contributors